

Simulation of a team of drones using the Webots simulator

Jakub Karczewski

Wojciech Fortuna

13.05.2026

1 Introduction and review of related research

1.1 Domain analysis

Search and rescue (SAR) operations are conducted in order to locate and assist missing persons in potentially dangerous environments. One of the most challenging scenarios for such operations is mountainous terrain.

Mountain regions are characterized by complex topography, including ridges, valleys and steep slopes. These features significantly limit accessibility and visibility. Ground-based search is often slow, physically demanding, and sometimes even dangerous for rescue teams. Additionally, environmental conditions such as weather, vegetation, and lack of infrastructure may further hinder the search process.

In mountainous areas, even relatively short distances on a map may correspond to difficult or impassable routes in practice. This makes systematic search particularly challenging and often requires dividing the terrain into smaller regions that can be explored independently.

Time plays a crucial role in search and rescue missions. The probability of finding a missing person decreases with time, which makes efficient search strategies extremely important. In many real-world cases, the first few hours of the search are the most critical.

In recent years, unmanned aerial vehicles (UAVs), commonly referred to as drones, have been increasingly used to support search operations. As shown in [goodrich2008uav], drones allow fast exploration of large areas and can access locations that are difficult to reach by humans.

Compared to ground-based methods, UAVs can significantly reduce the time required to scan large areas. They can also be deployed quickly and operate independently of road infrastructure. This makes them particularly useful in emergency situations.

However, the use of drones does not eliminate all challenges. One of the key limitations is restricted visibility caused by terrain. Even though drones operate in three-dimensional space, their sensors are often unable to detect objects hidden behind terrain obstacles such as hills or ridges. In practice, this means that even relatively close locations may remain unobserved due to terrain occlusions.

As a result, planning effective search trajectories becomes a non-trivial task, especially when multiple drones are involved.

1.2 Problem analysis

The main problem considered in this work is the coordination of multiple drones performing a search task in a mountainous environment.

The environment is represented by a terrain model, where elevation varies across the area. The terrain model is assumed to be known in advance. This means that all geometric properties of the environment, including elevation and visibility relations, are available before trajectory planning begins.

This allows the problem to be formulated as an offline planning task, where trajectories are computed based on complete knowledge of the environment.

Visibility between two points depends on the terrain shape. The importance of such visibility constraints has been discussed in computational geometry [deberg2008cg]. In practice, this implies that the ability of a drone to observe a region cannot be determined solely based on distance.

The searched person is assumed to be located at an unknown position. Instead of a deterministic location, the position is modeled using a probability distribution over the terrain. This approach is commonly used in search theory [stone1975search] and allows incorporating prior knowledge about the likely location of the target.

For example, certain regions may be considered more probable than others due to terrain characteristics, accessibility, or last known position of the missing person.

Each drone follows a predefined trajectory. During the mission, drones do not change their behavior or adapt to new information. This assumption simplifies the problem and allows focusing on the quality of trajectory planning. As a consequence, the effectiveness of the search depends entirely on how well the trajectories are designed before the mission.

Detection of the target depends on two main factors: distance and visibility. A drone can detect the target only if it is sufficiently close and if there are no terrain obstacles blocking the line-of-sight.

This makes the problem inherently three-dimensional and strongly dependent on terrain geometry. In particular, the same trajectory may be effective in one region of the terrain and ineffective in another.

The goal of the problem is to design trajectories for multiple drones such that the expected time required to detect the target is minimized.

1.3 Related approaches

The described problem is related to several research areas, including coverage path planning, probabilistic search, and multi-agent systems.

Coverage path planning focuses on systematic exploration of a given region. As discussed in [choset2001coverage], classical approaches such as grid-based methods and sweeping patterns are effective when the objective is to ensure complete coverage of the environment.

However, in the considered problem, full uniform coverage is not the primary goal. Instead, the objective is to minimize the expected detection time of a target whose location is described by a probability distribution. Consequently, different regions of the terrain have different importance, and uniform coverage strategies may be inefficient if they do not prioritize high-probability areas. Moreover, detection depends not only on visiting a region, but also on distance and line-of-sight conditions.

Another important area is probabilistic search, where the target location is modeled using a probability distribution [stone1975search]. In such approaches, search strategies are designed

to prioritize regions with higher likelihood of containing the target. This aligns more closely with the objective considered in this work.

However, many probabilistic search models assume simplified sensing conditions or do not explicitly account for terrain-dependent visibility. In the presence of terrain obstacles, visiting a region does not necessarily imply that the target can be detected. Detection depends explicitly on distance and line-of-sight conditions, which must be satisfied simultaneously.

Coordination of multiple agents in search and coverage tasks has also been widely studied. For example, [schwager2011distributed] investigates distributed control strategies for multi-robot systems. The use of multiple drones allows parallel exploration of the environment, which can significantly reduce search time.

At the same time, multi-agent coordination introduces additional challenges, such as avoiding redundant coverage and ensuring efficient allocation of search effort between agents.

In summary, each of the discussed approaches addresses a different aspect of the problem, including coverage, uncertainty, and multi-agent coordination.

1.4 Research goals

The objective of this project is to analyze and simulate the problem of coordinated search using a team of drones in a three-dimensional terrain.

The main goals of the project are:

- to model the search environment using a terrain representation,
- to incorporate visibility constraints resulting from terrain geometry,
- to model uncertainty of the target location,
- to design trajectory planning strategies for multiple drones,
- to minimize the expected detection time,
- to evaluate the approach in a simulation environment.

2 Problem formulation and proposed solution

In this section, a formal model of the considered multi-drone search problem is introduced. The formulation includes the terrain representation, the drone motion model, the probabilistic description of the target location, and the detection conditions.

Based on these components, the objective function is defined as the expected detection time. The problem is treated as an offline trajectory planning task under geometric and kinematic constraints.

2.1 Problem formulation

2.1.1 Environment model

The environment is represented by a three-dimensional terrain model given in the form of a height function

$$h : \mathbb{R}^2 \rightarrow \mathbb{R}. \quad (1)$$

For any horizontal position $(x, y) \in \mathbb{R}^2$, the value $h(x, y)$ defines the elevation of the terrain at that point.

The terrain model is assumed to be known before the planning stage. This allows determining geometric properties of the environment, including visibility relations between points in space.

In particular, visibility between two points depends on whether the line segment connecting them intersects the terrain surface.

2.1.2 Drone model

A team of N drones is considered. Each drone $i \in \{1, \dots, N\}$ is described by its trajectory

$$x_i(t) \in \mathbb{R}^3, \quad t \in [0, T], \quad (2)$$

where T denotes the time horizon of the mission.

Each drone is assigned an initial position

$$x_i(0) = x_i^0 \in \mathbb{R}^3, \quad (3)$$

which is known and fixed before the mission begins.

The motion of each drone is subject to kinematic constraints. In particular, the velocity is bounded by

$$\|\dot{x}_i(t)\| \leq v_{\max}, \quad \forall t \in [0, T]. \quad (4)$$

Additionally, drones operate above the terrain surface, which implies

$$z_i(t) \geq h(x_i(t), y_i(t)). \quad (5)$$

Trajectories are planned offline and remain fixed during the mission.

2.1.3 Detection model

Detection of the target depends on both distance and visibility.

Let $x_i(t)$ denote the position of drone i at time t . The detection function is defined as

$$D_i(t, y) = \begin{cases} 1 & \text{if } \|x_i(t) - y\| < d_{\max} \wedge \text{LoS}(x_i(t), y) = 1, \\ 0 & \text{otherwise.} \end{cases} \quad (6)$$

The function $\text{LoS}(x, y)$ indicates whether there exists a direct line-of-sight between positions x and y , i.e., whether the line segment connecting them is not obstructed by the terrain.

2.1.4 Detection time

For a given target position y , the detection time is defined as

$$T(y) = \min \{t \geq 0 : \exists i \in \{1, \dots, N\}, D_i(t, y) = 1\}. \quad (7)$$

If the target is not detected within the time horizon T , the detection time is set to T .

2.1.5 Objective function

The objective of the problem is to minimize the expected detection time over all possible target locations. Formally, this is expressed as

$$\min_{\{x_i(t)\}} \mathbb{E}_{y \sim p(y)} [T(y)]. \quad (8)$$

The objective function serves as the main performance measure used to evaluate the quality of planned trajectories.

2.2 Proposed algorithms

In order to address the trajectory planning problem formulated above, two complementary algorithms are proposed. The first one is a greedy constructive method that generates feasible trajectories based on local utility. The second one is a genetic algorithm that performs global optimization over the space of multi-drone trajectories.

The combination of these two approaches allows both efficient initialization and further improvement of solution quality.

2.2.1 Greedy multi-drone trajectory planning

The first proposed method is a greedy algorithm that incrementally constructs trajectories for all drones. The main idea is to iteratively select observation points that maximize the ratio of expected information gain to travel time.

Discretization The continuous terrain is discretized into a finite set of cells

$$G = \{c_1, c_2, \dots, c_M\},$$

where each cell c_j is associated with a probability p_j of containing the target.

Additionally, a finite set of candidate observation points is defined:

$$V = \{v_1, v_2, \dots, v_K\}.$$

For each observation point v_k , a visibility set $S_k \subseteq G$ is precomputed, containing all cells that can be detected from v_k , taking into account both distance constraints and line-of-sight conditions.

Utility function At each step, the algorithm evaluates candidate observation points based on their marginal contribution.

Let $U \subseteq G$ denote the set of cells that have not yet been observed. For a drone currently located at position u , the travel time to a candidate point v is defined as

$$\tau(u, v) = \frac{\|u - v\|}{v_{\max}}.$$

The marginal gain of visiting point v is defined as

$$\Delta(v) = \sum_{c_j \in S_v \cap U} p_j,$$

which represents the total probability mass of newly observable cells.

The selection criterion is given by

$$\text{score}(v) = \frac{\Delta(v)}{\tau(u, v) + \varepsilon},$$

where $\varepsilon > 0$ is a small constant to avoid division by zero.

Algorithm description The algorithm operates iteratively over all drones. At each iteration, it selects the drone and observation point that maximize the score function.

Formally, the following pair is selected:

$$(i^*, v^*) = \arg \max_{i, v} \text{score}_i(v),$$

subject to feasibility constraints, such as the remaining time budget.

After selecting the best move:

- the observation point v^* is appended to the trajectory of drone i^* ,
- the drone's position and time are updated,
- all cells in S_{v^*} are removed from the set U .

The process continues until no further feasible moves are possible.

Discussion The greedy algorithm is computationally efficient and naturally prioritizes regions with high probability of containing the target. It also avoids redundant coverage by removing already observed cells from consideration.

However, due to its myopic nature, the algorithm may produce suboptimal solutions, especially in complex terrains where long-term planning is required. For this reason, it is used primarily as a baseline method and as an initialization for further optimization.

2.2.2 Genetic algorithm for trajectory optimization

To improve upon the solutions generated by the greedy method, a genetic algorithm (GA) is proposed. The GA performs a global search over the space of possible multi-drone trajectories.

Solution representation Each individual in the population represents a complete set of trajectories for all drones. A solution is encoded as a sequence of observation points with an assignment to drones:

$$\pi = \{(v_1, i_1), (v_2, i_2), \dots, (v_L, i_L)\},$$

where $v_k \in V$ and $i_k \in \{1, \dots, N\}$ indicates the drone assigned to the observation point.

The order of elements determines the sequence in which observation points are visited.

Fitness function The fitness of an individual is defined based on the expected detection time. For each cell c_j , the time of first observation $t(c_j)$ is determined from the trajectories.

The objective function is given by

$$J = \sum_{j=1}^M p_j t(c_j),$$

which corresponds to the expected detection time.

The fitness function used in the genetic algorithm is defined as

$$f = -J,$$

so that lower detection times correspond to higher fitness values.

Initialization The initial population consists of:

- solutions generated by the greedy algorithm,
- randomly generated solutions,
- perturbed versions of greedy solutions.

This hybrid initialization improves both solution quality and population diversity.

Genetic operators The following operators are used to evolve the population.

Mutation Mutation introduces local modifications to trajectories. The following mutation types are considered:

- swapping two observation points,
- reassigning a point to a different drone,
- inserting a new observation point,
- removing an existing point,
- reversing a segment of a trajectory.

These operations allow exploration of the solution space while preserving feasibility.

Crossover Crossover combines parts of two parent solutions to produce offspring. A simple segment-based crossover is used:

- a subsequence of observation points is selected from one parent,
- the remaining points are filled using the second parent while preserving order.

Constraint handling After each genetic operation, solutions are checked for feasibility. If necessary, repair procedures are applied:

- trajectories exceeding the time horizon are truncated,
- infeasible transitions are removed or corrected.

Algorithm procedure The genetic algorithm follows the standard evolutionary scheme:

1. initialize population,
2. evaluate fitness of all individuals,
3. repeat until termination:
 - select parents,
 - apply crossover and mutation,
 - repair infeasible solutions,
 - evaluate offspring,
 - update population.

Discussion The genetic algorithm enables exploration of a large and complex solution space and can significantly improve upon greedy solutions. It is particularly suitable for multi-drone coordination, as it naturally handles assignment and ordering decisions.

However, it is computationally more expensive than the greedy approach and requires careful design of representation and operators.

2.3 Simulation model and implementation

This section describes the developed simulation model and the main software components used in the implementation of the multi-drone search system.

2.3.1 Simulation environment

The simulation environment was implemented using the Webots simulator. Webots provides a three-dimensional environment with support for programmable robot controllers, scene modeling, and visualization of autonomous agents.

The project uses a minimal Webots world stored in the `worlds` directory. The simulation world contains the terrain representation and drone objects used to execute generated waypoint trajectories. The Webots-specific controller is placed in a separate controller directory, following the structure required by Webots.

The trajectory planning modules operate outside the simulator and generate waypoint sequences using the terrain model, probability map, visibility model, and selected planning algorithm. The generated trajectories are exported to a JSON file and then loaded by the Webots drone controller.

The communication between the planning framework and the Webots simulation is therefore file-based. The generated trajectory file has the following structure:

$$\{ "drone_0" : [[x_1, y_1, z_1], [x_2, y_2, z_2], \dots] \}$$

with analogous entries for additional drones.

During simulation, each drone follows the assigned sequence of waypoints. After reaching a waypoint within a predefined tolerance, the controller switches to the next point. This provides a simple execution layer for trajectories generated by the greedy planner or the genetic algorithm.

The current Webots integration is intentionally minimal and focuses on validating the simulation framework rather than conducting experimental evaluation. Detailed comparisons of trajectory planning methods are left for the experimental part of the report.

2.3.2 Terrain representation

The environment is represented using a continuous terrain height function

$$h(x, y),$$

which defines terrain elevation for every horizontal position (x, y) .

For simulation purposes, the terrain is discretized into a finite grid of cells. Each cell stores:

- horizontal coordinates,
- terrain elevation,
- target probability value.

The current implementation uses synthetic mountainous terrain generated from smooth analytical elevation functions.

The discretized terrain representation simplifies visibility analysis, probability modeling, and trajectory planning.

In addition to the internal terrain representation used by the planning algorithms, the generated terrain is also synchronized with the Webots simulation environment in order to provide consistent three-dimensional visualization of the search area.

The terrain geometry is automatically exported from the Python simulation layer to a dedicated Webots PROTO object represented as a polygonal mesh. The current implementation uses an IndexedFaceSet mesh representation to ensure proper compatibility with the Webots rendering system and coordinate conventions.

Terrain coordinates are converted from the internal simulation representation:

$$[x, y, h(x, y)]$$

to the Webots coordinate convention:

$$[x, h(x, y), y].$$

This approach allows the trajectory planning algorithms, visibility analysis modules, and Webots visualization layer to operate on geometrically consistent terrain representations.

2.3.3 Visibility model

One of the key aspects of the considered problem is terrain-dependent visibility.

Visibility between a drone and a target position is evaluated using a line-of-sight (LoS) model.

The visibility condition is approximated by sampling intermediate points along the line segment connecting the drone and the target. For every sampled point, terrain elevation is compared with the altitude of the visibility ray.

If the terrain intersects the viewing ray at any sampled point, the target is considered invisible.

Formally, visibility is defined as

$$\text{LoS}(x, y) = 1$$

when the line segment connecting positions x and y is not obstructed by terrain.

This approach provides a computationally efficient approximation of terrain-aware visibility.

2.3.4 Detection model

Detection depends on both distance and visibility conditions.

A target can be detected only if:

- the target remains within the detection radius,
- direct line-of-sight exists between the drone and the target.

The target position is modeled slightly above the terrain surface in order to simulate a standing person and avoid numerical issues related to terrain intersection.

2.3.5 Probability map

The uncertainty of the target location is represented using a probability distribution over the terrain grid.

Each terrain cell is assigned a probability value describing the likelihood of the target being located in that region.

In the current implementation, the distribution is generated using Gaussian hotspots centered around selected terrain regions. This allows modeling prior information about likely target locations.

The probabilities are normalized such that

$$\sum_{j=1}^M p_j = 1.$$

2.3.6 Trajectory planning implementation

The current implementation includes a trajectory planning framework for cooperative multi-drone search operations.

The environment is discretized into:

- terrain cells,
- candidate observation points.

For every candidate observation point, a visibility set is precomputed:

$$S_k \subseteq G,$$

where S_k contains all cells observable from observation point v_k .

This preprocessing step significantly reduces the computational cost during trajectory planning.

At the current stage of the project, two trajectory planning approaches have been implemented:

- a greedy trajectory planner,
- a genetic algorithm for trajectory optimization.

The greedy planner iteratively selects observation points maximizing the ratio between:

- expected probability gain,

- travel cost.

Candidate moves are evaluated using the score function

$$\text{score}(v) = \frac{\Delta(v)}{\tau(u, v) + \varepsilon},$$

where:

- $\Delta(v)$ denotes the probability mass of newly observable cells,
- $\tau(u, v)$ denotes travel time,
- ε is a small constant preventing division by zero.

The current implementation supports both:

- single-drone planning,
- cooperative multi-drone planning.

In the multi-drone setting, all drones share information about already observed cells in order to reduce redundant coverage.

Additionally, a genetic algorithm framework was implemented for optimization of multi-drone trajectories.

In the implemented genetic algorithm, each solution is represented as a chromosome containing a sequence of observation assignments:

$$(drone_id, candidate_id),$$

where each gene specifies which drone visits a selected observation point.

This representation allows simultaneous optimization of:

- assignment of observation points to drones,
- ordering of observation points,
- overall trajectory structure.

The initial population is generated randomly from the set of candidate observation points.

The fitness function combines:

- covered target probability mass,
- travelled distance penalty,
- mission time constraints.

Solutions violating the time budget constraints are repaired by iteratively removing observation points from the chromosome.

The implemented evolutionary process includes:

- tournament selection,
- one-point crossover,
- mutation of drone assignments and observation points,
- elitism preserving the best individuals between generations.

The modular architecture of the framework allows further extension with additional optimization methods and more advanced evolutionary operators.

2.3.7 Mission simulation and Webots integration

The developed framework includes a lightweight mission simulation and execution layer integrated with the Webots simulator.

The overall system architecture is divided into four main layers:

- trajectory planning layer,
- mission simulation layer,
- execution layer,
- visualization layer.

The trajectory planning layer operates outside the Webots simulator and is responsible for generating waypoint trajectories using the terrain model, probability map, visibility analysis, and trajectory planning algorithms.

The generated trajectories are exported to external JSON files in the following form:

$$\{ "drone_0" : [[x_1, y_1, z_1], [x_2, y_2, z_2], \dots] \}.$$

The exported trajectories are then loaded by Webots drone controllers responsible for waypoint execution inside the simulation environment.

The mission simulation layer is implemented using the `SimulationManager` component. This layer is responsible for:

- simulation time management,
- updating drone positions,
- target detection verification,
- tracking observed terrain cells,
- mission termination handling.

The mission execution process terminates when at least one of the following conditions is satisfied:

- the target is detected,
- the mission time budget is exceeded,
- no significant unexplored probability regions remain,
- all generated trajectories are completed.

The mission state is synchronized with the Webots execution layer using external JSON status files containing:

- mission completion status,
- target detection status,
- mission termination reason,

- mission statistics,
- target visualization information.

The Webots execution layer is intentionally lightweight. Drone motion is implemented using simplified kinematic waypoint following by directly updating the robot translation field inside the simulator. The current implementation does not include advanced flight dynamics, collision avoidance, low-level quadcopter control, or online replanning mechanisms.

The Webots environment is therefore used primarily as:

- a trajectory execution framework,
- a visualization environment,
- a lightweight mission playback system.

The current implementation additionally supports synchronization of the generated terrain geometry with the Webots environment. The terrain generated by the Python simulation layer is automatically exported to a dedicated Webots PROTO object represented as a polygonal mesh.

The simulation environment additionally contains a visual representation of the target object. The true target position is synchronized with Webots using a dedicated target status file and a separate visualization controller responsible for updating the target position inside the simulation environment.

Importantly, the trajectory planning algorithms do not have access to the true target position. The true target location is known only by the mission simulation and visualization layers. This separation preserves the probabilistic nature of the search problem and prevents the planning algorithms from exploiting ground-truth target information.

The implemented integration framework provides a modular connection between the external Python-based planning algorithms and the Webots simulation environment while maintaining low implementation complexity and high extensibility for future development.

2.3.8 System architecture

The implemented framework consists of several interconnected modules responsible for terrain generation, probability modeling, visibility evaluation, detection checking, trajectory planning, mission simulation, and visualization.

The terrain model and probability map provide input data for the trajectory planning algorithms. The visibility and detection modules are used to determine which terrain cells can be observed from candidate drone positions.

Based on this information, the trajectory planning layer generates waypoint trajectories for multiple drones. The generated trajectories are exported to external JSON files and executed inside the Webots simulation environment.

The framework additionally includes a mission simulation layer responsible for:

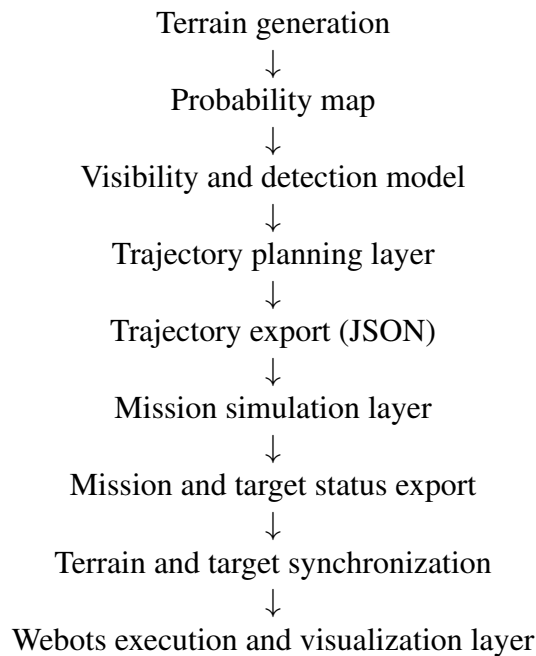
- mission time management,
- target detection verification,
- observed terrain tracking,
- mission termination handling.

The simulation framework additionally supports synchronization of terrain geometry and target visualization with the Webots environment. Dedicated export modules are used to transfer terrain meshes, drone trajectories, mission status information, and target visualization data between the Python-based simulation layer and the Webots execution layer.

The Webots simulation layer is responsible for:

- terrain visualization,
- waypoint execution,
- drone visualization,
- target visualization,
- mission playback.

The implemented system architecture is illustrated below:



The modular architecture simplifies further development of the framework and allows independent extension of the planning, simulation, execution, and visualization components.

2.3.9 Software libraries

The implementation was developed in Python and uses several software tools and libraries supporting numerical computation, visualization, and simulation.

The following software components were used:

- Webots simulator,
- Python,
- NumPy,
- Matplotlib,
- built-in Python JSON library.

Python was used as the primary programming language for implementing the terrain model, visibility analysis, detection model, probability map representation, and trajectory planning algorithms.

NumPy was used for numerical computations, vector operations, and manipulation of three-dimensional coordinates. Matplotlib was used for visualization, debugging, and generation of plots during the development process.

The Webots simulator was used as the simulation and visualization environment for executing drone trajectories generated by the planning algorithms.

Drone trajectories were exported from the planning layer to the simulation environment using JSON files, enabling lightweight integration between the external Python-based planning framework and the Webots execution layer.

The built-in Python JSON library was used for serialization and exchange of waypoint trajectories between the planning algorithms and the Webots controllers.

2.4 Algorithm pseudocode

Algorithm 1: Line-of-sight visibility evaluation

Input: Drone position x , target position y , terrain height function h , number of samples K

Output: Visibility value $\text{LoS}(x, y)$

```

1 for  $k \leftarrow 1$  to  $K - 1$  do
2    $\alpha \leftarrow k/K$ ;
3    $p \leftarrow (1 - \alpha)x + \alpha y$ ;
4    $z_{\text{terrain}} \leftarrow h(p_x, p_y)$ ;
5   if  $z_{\text{terrain}} > p_z$  then
6     return 0;
7   end
8 end
9 return 1;
```

Algorithm 2: Greedy multi-drone trajectory planning

Input: Drones initial positions x_i^0 , candidate observation points V , terrain cells G , probabilities p_j , time horizon T

Output: Trajectories for all drones

```

1 Initialize trajectories with initial drone positions;
2 Initialize current drone times  $t_i \leftarrow 0$ ;
3 Initialize unobserved cells  $U \leftarrow G$ ;
4 foreach candidate observation point  $v_k \in V$  do
5   | Precompute visibility set  $S_k$ ;
6 end
7 while  $U \neq \emptyset$  do
8   |  $bestScore \leftarrow 0$ ;
9   |  $bestMove \leftarrow \emptyset$ ;
10  foreach drone  $i$  do
11    | foreach candidate observation point  $v_k \in V$  do
12      | Compute travel time  $\tau(x_i, v_k)$ ;
13      | if  $t_i + \tau(x_i, v_k) > T$  then
14        |   | continue;
15      | end
16      |  $\Delta(v_k) \leftarrow \sum_{c_j \in S_k \cap U} p_j$ ;
17      |  $score \leftarrow \frac{\Delta(v_k)}{\tau(x_i, v_k) + \epsilon}$ ;
18      | if  $score > bestScore$  then
19        |   |  $bestScore \leftarrow score$ ;
20        |   |  $bestMove \leftarrow (i, v_k)$ ;
21      | end
22    | end
23  end
24  if  $bestMove = \emptyset$  then
25    | break;
26  end
27  Append selected point to the selected drone trajectory;
28  Update selected drone position and time;
29  Remove newly observed cells from  $U$ ;
30 end
31 return trajectories;

```

Algorithm 3: Multi-drone genetic algorithm

Input: Drones initial positions x_i^0 , candidate observation points V , visibility sets S_k , probabilities p_j , population size P , number of generations G

Output: Optimized multi-drone trajectories

- 1 Initialize population of P random chromosomes;
- 2 **foreach** *chromosome* π **in** *population* **do**
- 3 | $\pi \leftarrow \{(i_1, v_1), (i_2, v_2), \dots, (i_L, v_L)\};$
- 4 **end**
- 5 **for** $g \leftarrow 1$ **to** G **do**
- 6 | **foreach** *individual* π **in** *population* **do**
- 7 | Decode π into trajectories for all drones;
- 8 | Repair π if any drone exceeds the time horizon;
- 9 | Determine observed cells using visibility sets S_k ;
- 10 | $P_{\text{cov}} \leftarrow \sum_{c_j \in \text{observed}} p_j$;
- 11 | Compute total trajectory distance d_{total} ;
- 12 | $\text{fitness}(\pi) \leftarrow P_{\text{cov}} - \lambda d_{\text{total}}$;
- 13 | **end**
- 14 | Preserve elite individuals;
- 15 | **while** *new population is not full* **do**
- 16 | Select two parents using tournament selection;
- 17 | Apply one-point crossover;
- 18 | Mutate drone assignments or observation points;
- 19 | Repair infeasible offspring;
- 20 | Add offspring to new population;
- 21 | **end**
- 22 | Replace old population with new population;
- 23 **end**
- 24 Return the best chromosome decoded into drone trajectories;
